

A practical applications guide to machine learning regression models in psychology with Python

Nataša Kovač^a, Kruna Ratković^{a,*}, Hojjatollah Farahani^b, Peter Watson^c

^a Department of Mathematics, Faculty of Applied Sciences, University of Donja Gorica, Oktoih 1, Podgorica, 81000, Montenegro

^b Department of Psychology, Tarbiat Modares University, Jalal AleAhmed, Tehran, 14115-111, Nasr, Iran

^c MRC Cognition and Brain Sciences Unit, Cambridge University, 15 Chaucer Road, Cambridge, CB2 7EF, United Kingdom

ARTICLE INFO

Keywords:

Python
Machine learning
Psychological applications
Regression models
Tutorial

ABSTRACT

This guide presents a detailed overview of the most used machine learning (ML) techniques for psychologists who may not be familiar with advanced statistical methods, algorithms, or programming. Recognizing the growing interest in using data-driven approaches within psychological research, this guide describes applying ML techniques to investigate complex psychological phenomena. The paper covers the spectrum of algorithms, including decision trees, random forests, gradient boosting, stochastic gradient boosting, and XGBoost, highlighting their concepts and practical applications in psychology. Aiming to bridge the gap between theoretical understanding and practical performance, this paper offers step-by-step instructions on data preprocessing, correlation exploration, feature selection, and model evaluation within the Python programming environment. Readers are offered the necessary tools to apply ML in their research through explanations, examples, and visualization.

1. Introduction

In the dynamic field of psychology, the emergence of ML represents a significant change, blending the areas of computer science with human behavior and cognitive processes. ML, as a subset of artificial intelligence, involves the development of algorithms that enable computers to learn from and make predictions or decisions based on data. For psychologists, this integration offers a new lens through which to view complex human patterns and behaviors, transcending traditional analytical boundaries. By utilizing ML, psychologists can refine their methodologies, enhance diagnostic accuracy, and tailor interventions more effectively to meet individual needs.

The potential of ML within psychological research is profound and multifaceted. It empowers researchers to filter through extensive amounts of data, from behavioral observations to neurological images, to uncover underlying patterns and insights. This ability is especially important to identify nuanced yet impactful details that might escape human observation. ML algorithms can handle the complexities and nuances of psychological data, facilitating a deeper understanding of mental health conditions, cognitive processes, and social interactions. By leveraging these advanced analytical tools, psychological research can move beyond surface-level observations to unearth nuanced insights

that drive more personalized and impactful interventions.

Recognizing the growing interest in using data-driven approaches within psychological research, this tutorial aims to bridge the gap between theoretical understanding and practical performance. The primary objective of this guide is to provide a comprehensive overview of the most utilized ML techniques adapted specifically for psychologists who may not have extensive knowledge of advanced statistical methods, algorithms, or programming. Through detailed step-by-step instructions on data preprocessing, correlation exploration, feature selection, and model evaluation within the Python programming environment, this tutorial equips psychologists with the necessary tools to apply ML methodologies in their research.

By the end of this tutorial, psychologists will have a fundamental understanding of ML algorithms, be able to perform data preprocessing, detect and interpret correlations, select appropriate features, and evaluate model performance using Python. This guide is designed to equip psychologists with the necessary skills to apply ML methodologies in their research, thereby enhancing their ability to derive more profound and actionable insights from their data, increasing the impact of their psychological research. Additionally, this tutorial will provide a focused case study on utilizing diverse supervised ML regression methods, Decision Tree (DT), Random Forest (RF), Gradient Boosting (GB),

* Corresponding author.

E-mail address: kruna.ratkovic@udg.edu.me (K. Ratković).

<https://doi.org/10.1016/j.metip.2024.100156>

Received 16 April 2024; Received in revised form 10 July 2024; Accepted 17 July 2024

Available online 3 August 2024

2590-2601/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Stochastic Gradient Boosting (SGB), and Extreme Gradient Boosting (XGBoost), to analyze a dataset that explores multiple aspects related to internal shame. This will enable a practical learning experience in applying various ML techniques.

This tutorial underscores the importance of integrating ML into psychological research to push the boundaries of what can be achieved with traditional methods alone. It not only broadens the scope of psychological inquiry but also enables more innovative and impactful research outcomes.

2. Role of machine learning in psychology

ML represents an advanced evolution of traditional statistical analysis, extending its capabilities far beyond pure data interpretation. While rooted in statistical theory, ML differentiates itself through its ability to learn from data, adapt to new information, and make predictions or decisions with minimal human intervention. This autonomous learning capability allows for handling complex datasets more efficiently, revealing concealed trends and insights that could be missed by conventional statistical methods. ML algorithms continually improve their performance as they are exposed to more data, making them exceptionally advantageous in dynamic environments where conditions and variables constantly change. This makes ML a powerful tool not just for predictive analytics but also for automating decision-making processes and creating innovative solutions across various fields.

Psychology has been slow in adopting ML as a method for analyzing experimental data (Orrù et al., 2020) with, psychological experimental data largely analyzed using p-values and, more recently, effect size measures. Bennett et al. (2022) contrasted the assumptions and uses of ML with those of traditional analyses.

Classical statistical modeling was designed for data with a few dozen input variables and sample sizes that would be considered small to moderate today. Classical inference struggles to capture the more complex relationships associated with larger numbers of input variables. Consequently, statistical inferences become less precise (Bzdok et al., 2018).

Breiman (2001b) noted that classical statistical approaches assume that data are generated by a given stochastic data mode with evaluation based on how well the model fits the data. Statistical inference based on data modeling has been the standard de facto procedure in the analysis of scientific experiments since 1940.

Inference creates a mathematical model of the data-generation process to formalize understanding or test a hypothesis about how the system behaves. Statistical methods have a long-standing focus on inference, which is achieved through the creation and fitting of a project-specific probability model. The model allows us to compute a quantitative measure of confidence that a discovered relationship describes a *true* effect that is unlikely to result from noise (Orrù et al., 2020). Ioannidis et al. (2011) state that measures for inference in classical statistics typically including p-values and effect sizes may lack reproducibility.

The application of ML techniques can help psychology become a more predictive science (Yarkoni and Westfall, 2017). While traditional psychological research focuses on identifying how predictor variables causally influence outcome variables, ML models prioritize achieving the highest possible accuracy, without bias, in predicting outcome variables (Rosenbusch et al., 2021). ML techniques enable enhanced sensitivity and improved predicting ability over conventional explanatory methods (Koul et al., 2018). In particular, the ML approach treats the data as unknown and is mainly focused on predictive accuracy. Prediction aims at forecasting unobserved outcomes or future behavior. Prediction is also addressed in classical statistics but with models that are usually constrained by strong assumptions (e.g., linear regression and logistic regression). ML models are more focused on prediction and are *model agnostic*. In most data analyzed with ML models similar levels of predictive accuracy may be achieved using models that rely on very

different assumptions (e.g., Support Vector Machine, Naive Bayes, KNN, RF). In ML models, prediction is achieved by using general purpose learning algorithms to find patterns in often large and highly complex datasets. ML methods are particularly helpful when one is dealing with datasets in which the number of input variables exceeds the number of subjects, as opposed to datasets where the number of subjects is greater than that of the input variables.

ML makes minimal assumptions about the data-generating systems. They can be effective even when the data are gathered without a carefully controlled experimental design and in the presence of complicated non-linear interactions. However, even when these models have high predictive accuracy, the lack of an explicit model can make ML solutions difficult to directly relate to existing psychological knowledge. The boundary between statistical inference and ML is fuzzy and methods originally developed in statistics are included in the ML toolbox. For example, linear and logistic regressions, hierarchical clustering and Principal components analysis are routinely included in all ML packages (Orrù et al., 2020). Statistics requires us to choose a model that incorporates our knowledge of the system, and ML to determine a predictive algorithm by relying on its empirical capabilities.

The intersection of ML with variables of uncertain reliability can present challenges in the context of model selection. While some researchers may rely on a single statistical approach, it is beneficial to evaluate models of varying complexity, ranging from linear models (e.g., linear or logistic regression), to more complex nonlinear models like random forests or boosting (Jacobucci and Grimm, 2020). This comparative analysis can explain the nature of the relationships present in the data. Using only linear models can result in missing potential nonlinear dynamics, and using only nonlinear methods may lead to an overestimation of the complexity in the relationships. The selection of the most appropriate model is linked to how variables are measured. A thorough understanding of the reliability of both predictor and outcome variables is crucial. It can indicate whether the introduction of new measurement techniques can correct underperformance and explain why a linear model might be preferable in cases where nonlinear or interactive effects were initially anticipated.

3. Setting up the Python environment

Python is a programming language that has gained widespread popularity for its simplicity, readability, and flexibility. Especially beneficial for psychologists, Python offers an accessible entry point into programming due to its straightforward syntax, which is similar to everyday English. This feature significantly reduces the learning process, enabling professionals from non-technical backgrounds to harness computational tools and statistical techniques with ease. Furthermore, Python's extensive libraries, such as **NumPy**, **SciPy**, and **pandas**, provide powerful resources for data analysis, visualization, and machine learning, making it an invaluable tool for conducting sophisticated psychological research and analysis. Its open-source nature also fosters a collaborative environment where psychologists can share scripts, models, and findings, thereby enhancing the collective knowledge and capabilities of the field.

If you do not have an environment that supports working in Python, it is necessary to install it first. The simplest way is to visit the website <https://www.anaconda.com/> and install the Anaconda environment. Anaconda offers easy access to a large number of different open-source benefits, and most importantly, it also has access to Jupyter notebooks within its platform. When downloading, select the operating system you are using on your computer, and after a few minutes, when the file download is complete, install the complete software package. For more details you can check the installation tutorial (Rolon-Mérette et al., 2016).

Start Anaconda Navigator by typing its name in the Search field of the Windows system, or run it in the usual way, as you do with any other application. Launch your Jupyter Notebook from the screen that appears

in front of you, and you will see the environment with folder structure. On the top of the screen is a button labelled *New*. When you click it, choose option *Python 3 kernel* which will open a blank notebook. Now you are ready to work in Python.

Jupyter Notebook is a very popular interactive tool for running Python code in a web browser (Toomey, 2016). With the help of Jupyter Notebook, it is possible to create interactive notebooks that contain program code, visualization, and the necessary output in one file. The main advantage of Jupyter Notebook is that users can immediately see the effects of changes after changing the program code. Jupyter Notebook allows editing an already executed cell if a syntax error is detected or if it is necessary to correct the semantics of the code. As all program elements and outputs are united in one file, distribution, storage, and sharing of program code is significantly simplified. In addition, Jupyter Notebook is supported with a suite of tools and packages that make it easier for users to write and execute algorithms, thus enabling it to become one of the most widespread environments for Python programming in a large number of disciplines and for different purposes.

A Python package (library) is a set of modules that are interconnected to provide certain functionality or are created to solve tasks in a specific domain. The main advantage of the package is that, in addition to facilitating the organization, storage and sharing of code, its elements can be reused, thus ensuring the construction of complex and robust applications with significantly increased productivity and a simplified Python application development process. The existence of a large number of functional packages is the reason for the wide usability of Python. You can find a list of existing packages on the Python repository known as the Python Package Index (<https://pypi.org/>).

To use a library, it must first be installed. Installation is usually done by calling a package manager such as **pip** or **conda**. When the program code requires the use of a package or module, it is necessary to first import it via the **import** command, whereby all the package's functionalities become available within the program. More details about the programming elements of Python and its libraries can be found in a large number of available books and tutorials (Lutz, 2001; Martelli et al., 2023).

4. Methodology

Application of ML techniques is a process involving data preprocessing, selecting appropriate algorithms, and evaluating model performance. To ensure the reliability and validity of ML models the crucial step is data preprocessing. During this stage, inaccuracies in datasets are detected and corrected. Common techniques to improve data quality include imputing missing values, feature encoding, feature scaling, and outlier detection.

The choice of algorithms depends on their suitability for regression tasks and their ability to handle complex data structures. While both supervised and unsupervised learning techniques are available, our focus was on supervised learning techniques. Unsupervised learning methods, such as Principal Component Analysis (PCA) and clustering algorithms, are often used for feature reduction, data visualization, and uncovering hidden patterns within data without predefined labels. For this study, we selected supervised learning algorithms due to their robustness and accuracy in regression tasks. Algorithms such as DT, RF, GB, SGB, and XGBoost were chosen based on their performance in previous studies, ease of implementation in Python, and their interpretability. These characteristics made them well-suited for the regression tasks at hand.

The dataset needs to be split into training and testing sets (usually with a 80-20 ratio) to ensure the model's ability to generalize to new data. Then, each model was trained using the training set and evaluated on the test set. To assess model performance, we employed metrics such as Mean Absolute Error, which measures the average magnitude of errors in predictions, Mean Squared Error, which emphasizes larger errors by squaring them before averaging, Mean Absolute Percentage Error

measuring the percentage of deviation from actual values, and the coefficient of determination, which indicates the proportion of variance in the dependent variable predictable from the independent variables. Hyperparameters for each algorithm were optimized using grid search to enhance performance.

For the practical application, we focused on a case study predicting Internal Shame (IS) based on several psychological factors. The data underwent preprocessing, including cleaning, encoding, and scaling. Following this, the chosen algorithms were trained with the training dataset. The models were then evaluated using the test dataset and mentioned performance metrics. Finally, SHAP values were used to interpret the influence of each feature on the prediction of IS. Figures representing the correlation matrix, pairplot, and SHAP values were generated to provide visual insights into the data and model performance.

5. Preprocessing data

Data cleaning and preparation are crucial stages in the analytical process, ensuring the reliability and validity of results (Kotsiantis et al., 2006; Famili et al., 1997). This involves identifying and rectifying inaccuracies such as incorrect, duplicated, or incomplete data within datasets. These processes improve data consistency, eliminate redundancy, and ensure data integrity, directly impacting the analytical outcomes and decision-making processes. Techniques such as outlier detection, imputation for missing values, and data standardization are commonly applied to enhance data quality. Maintaining clean data is integral for operational efficiency, reducing errors, and facilitating accurate and timely analyses. Hence, proper data cleaning not only supports effective data analysis but also significantly contributes to the overall success of data-driven projects and organizational strategies.

In order to implement these activities in Python, it is necessary that the **pandas** library has been loaded into your working environment and that your dataset has been loaded into the data frame on which you will perform further analysis. If your data are placed in a **csv** file, import the dataset in Python by calling **df = pd.read_csv('data.csv')**. In addition, if you plan to perform data scaling, it is necessary to additionally import the functions **MinMaxScaler** and **StandardScaler** from **sklearn.preprocessing**. For visualization and detection of outliers, additional libraries such as **numpy**, **seaborn**, and **matplotlib.pyplot** are necessary.

Handling missing values is an essential step in preparing data for analysis, as incomplete datasets can lead to skewed results and misinterpretations. The process begins with detecting these missing entries, typically using functions in Python's **pandas** library, which help identify and quantify the extent of missingness. Deciding how to address these gaps, through elimination, imputation, flagging or other methods depends on the dataset's characteristics and the analysis goals (Peeters et al., 2015).

In the process of data preparation for analysis and model building, it is necessary to carefully approach the filling of missing values and the selection of the scaling method. These choices depend on the individual attributes of the data set as well as the specific requirements imposed by the models intended for use. In particular, one must take into account the fact that after the original data have been changed by any form of imputation, returning to the original data set becomes impossible. When choosing potential imputation methodologies, it is necessary to consider and evaluate different techniques, which will ensure the identification of a method that is characterized not only by robustness but also by reducing the bias and variations inherent in different techniques. It is a good practice to compare the original data set with the newly formed set that is subjected to imputation, thus gaining insight into the reliability and efficiency of the selected imputation method, but also ensuring the seamless integration of imputation into the complete process of data analysis and modelling.

In Python, missing values are denoted by **NaN**. If your dataset has already been placed in a dataframe, which for example has the name **df**,

by simply printing `df` it is possible to get an insight into the **NaN** order in the columns of the dataset. However, this method is inconvenient if your dataset has many records. The **pandas** library treats **NaN** as a missing value and provides several useful functions for their manipulation. The most frequently used methods are `df.isnull()`, `df.info()`, `df.isna()`, `df.fillna()`, and `df.dropna()`. If your dataset is numerous, the simplest insight into the distribution of missing values will be obtained by counting the missing values in each column of `df`. This is achieved by calling the command `print(df.isnull().sum())`.

If you detected missing values with the previous command, you could decide either to discard the records containing them or to fill them with meaningfully defined artificially generated data. In the first case, if the desired option is to delete records with such data, it is necessary to call the `df.dropna()` function. In this way, records that have at least one missing value are removed. If you want to remove records in which all values are missing, it is necessary to define the function as `df.dropna(how='all')`.

When removing data, existing related data is also removed, which carries the risk of losing important information from the dataset. This is why it can be useful to fill in missing values. With imputation, no additional data is needed, but information is extracted from existing data. Missing numeric data in `df` columns that are of type `int64` or `float64` (i.e., numeric columns) can be replaced, for example, with the mean value, by calling the command `df[col].fillna(df[col].mean(), inplace=True)`. Similarly, for columns that contain categorical data, that is, they have the type `object` or `category`, the command `df[col].fillna(df[col].mode()[0], inplace=True)` will fill in missing values with the most frequent value, that is, with the mode.

Additionally, suppose there is a concern that your dataset contains duplicate data that violates the integrity of the analysis. In that case, it is possible to detect them by calling the `print(df.duplicated().sum())` command, which will print the number of records that represent duplicates. These records are easily removed with the `df.drop_duplicates()` command.

Categorical variables are qualitative data types that represent categories or groups, such as gender, nationality, or product type. These variables are often encoded to transform them into numerical formats that ML algorithms can interpret more effectively (Agresti, 2012). One common approach is one-hot encoding, where each category is represented as a binary vector with one *hot* or *active* state indicating the presence of that category, while all other elements remain *cold* or *inactive*. Another method is label encoding, where each category is assigned a unique integer. Although straight forward, label encoding can unintentionally introduce a numerical hierarchy or order that might not exist. Ordinal encoding is similar but is specifically used when the categorical variables have a natural order, such as in ratings like *high*, *medium*, and *low*. It is crucial to choose the right encoding technique based on the specific data and the requirements of the machine learning model to ensure accurate and effective outcomes.

The decision to use one-hot encoding or label encoding is determined by the characteristics of the data, specifically whether it is nominal or ordinal. Nominal data types, such as therapy types, occupation, or nationalities, categorize elements into distinct groups without any inherent order. They serve as labels without indicating any rank or hierarchy. In these instances, one-hot encoding is preferred because it eliminates the creation of unintended hierarchies, which is a common issue with linear models. This method involves creating a new column for each category value and marking the presence or absence of the feature with a 0 or 1 in each row.

Ordinal data types categorize elements while also arranging them in a meaningful sequence, although without consistent intervals between ranks. For such data, label or ordinal encoding is more appropriate as it preserves the sequence of the categories. Label encoding assigns a unique integer to each category based on alphabetical order, while ordinal encoding assigns numbers based on a predefined order, reflecting the rank or sequence inherent in the data.

Applying one-hot encoding to attributes with numerous categories can lead to a significant increase in dimensionality. To mitigate this, dimension reduction strategies such as Principal Component Analysis or L1 regularization can be employed, or alternatively, leveraging tree-based algorithms which cope well with higher dimensions (Shen and Huang, 2008; Zou et al., 2006; d'Aspremont et al., 2004; Vidaurre et al., 2013). Also, the extensive adoption of one-hot encoding can result in a sparse matrix, raising memory consumption issues. However, specific algorithms and libraries have been optimized to efficiently manage such sparse matrices.

The one-hot encoder is called in Python with `pd.get_dummies(df, columns = ['col'])` in order to fill column `col` with coded values. For binary encoding, to transform the `col` column in the dataset the library `category_encoders` has to be loaded and named as `ce`, and then the commands.

```
encoder = ce.BinaryEncoder(cols=['col'])
df = encoder.fit_transform(df['col'])
```

are called. Similarly, in the case of label encoding, it is necessary from `sklearn.preprocessing` to import the `LabelEncoder` function, and then with commands.

```
le = LabelEncoder()
df['col'] = le.fit_transform(df['col'])
```

to transform the existing dataset. When encoding of ordinal categorical data is done, the dictionary `order = {'key1': 1, 'key2': 2, 'key3': 3, 'key4': 4}` is first defined, so that each key (category) is manually assigned a numerical value. Please note that the number of keys does not have to be limited to four. After that, the mapping and replacement (`df['col'] = df['col'].replace(order)`) of the categorical values in the `col` column with the defined values from the variable `order` is performed.

Normalizing or scaling features is a technique aimed at harmonizing the range of different variables (Jadhav et al., 2023). This process involves transforming the values of numerical features so that they align within a specific scale, such as 0 to 1 or -1 to 1. By applying normalization, we mitigate the risk of disproportionate influence by certain features due to their scale, ensuring that each attribute contributes evenly to the analysis. Scaling techniques, such as Min-Max scaling and Standardization (Z-score normalization), are employed based on the distribution of data and the algorithm requirements. This step is crucial in enhancing the performance and accuracy of ML models, as it allows algorithms to converge faster and perform more effectively during training phases.

With certain ML algorithms, especially those that belong to the group of gradient descent-based algorithms, it is important to implement the data normalization procedure. For this purpose, the functions `MinMaxScaler` and `StandardScaler` are used. The function `MinMaxScaler(feature_range=(0, 1))` scales the data to a 0–1 range, while `StandardScaler()` scales data to have a mean of 0 and a standard deviation of 1. Both of these functions only work on columns that contain numerical data, so it is necessary to select them, as well as in the previous considerations.

You need to adapt the scaler to your data and then transform your numeric features using the built-in scaler. With ML and AI algorithms, datasets are separated into training and test datasets. It is common practice to fit a scaler to the training data and then use it to transform both the training and test data. Assuming that the training set is in the variable `X_train` and the test dataset in the variable `X_test`, the procedure for fitting and transforming training data is obtained with `X_train_scaled = scaler.fit_transform(X_train)`. Similarly, the command `scaler.transform(X_test)` will transform the testing data. It is usual for this procedure to end with the replacement of the original data

in **df** with scaled values. This is achieved by **df[features] = X_train_scaled**.

Outliers are data points that deviate significantly from other observations in a data set. Identifying and handling outliers appropriately in data analysis will ensure the reliability and validity of the results (Aguinis et al., 2013; Ghosh and Vogt, 2012). They can arise due to measurement errors, data entry errors, or natural variations in populations. Outliers can significantly impact statistical analyses and lead to misleading results. For example, they can skew the mean and standard deviation, leading to incorrect assumptions and interpretations. In regression analysis, outliers can disproportionately influence the slope and intercept of the regression line, affecting the overall model's accuracy.

For visual identification of outliers, boxplots (**sns.boxplot(x = df['feature_name'])**) and scatter plots (**plt.scatter(df['feature1'], df['feature2'])**) can be used. From the point of view of statistical methods, known statistical metrics such as the Z-score or the Interquartile Range (IQR) are useful. If you have opted for Z-score statistics, it is necessary to first load this function from the **scipy.stats** library. Statistics can only be calculated on numerical data, so it is necessary to select only columns with numerical data types. This is achieved with **df.select_dtypes(include = [np.number])**. Outliers are commonly defined as values that deviate from the expected value by more than 3 standard deviations. Since the direction of data distribution is not important for this type of analysis, we can use the absolute value, which

```
z_scores = np.abs(zscore(df.select_dtypes(include=[np.number])))
outliers = (z_scores > 3).any(axis=1)
```

gives the resulting commands.

The variable **outliers** can now be used to eliminate all outliers from **df** as follows **df_no_outliers = df[~outliers]**.

The interquartile range technique is often used in statistics to detect outliers, especially in data sets with roughly symmetrical distributions. For each numerical column in the dataset, it is necessary to calculate the value of the first (**Q1 = df.quantile(0.25)**) and third quartile (**Q3 = df.quantile(0.75)**), and then the value for the **IQR = Q3 - Q1**. Values that are more than 1.5 IQRs below the first quartile or 1.5 IQRs above the third quartile are declared outliers. In this way, all values that are considered outliers according to the IQR method are detected in the variable **outliers** with the command **outliers = (df < (Q1 - 1.5 * IQR)) | (df > (Q3 + 1.5 * IQR))**. Similar to the previous case, the generated variable can be used to eliminate all outlier values from the data set by calling the command **df[~outliers.any(axis=1)]**.

The influence of outliers on the analysis can be mitigated by their transformation instead of elimination. For example, it is possible to set predefined values of the maximum or minimum outliers or replace their values with calculated values based on the dataset, e.g., the median value.

6. Detecting correlations and feature selection

Correlation refers to a statistical measure that expresses the extent to which two variables change together. It takes values from -1 to 1 , where 1 indicates a perfect positive correlation and -1 indicates a perfect negative correlation. In the context of feature selection for ML models, understanding correlation will help to identify the relationships between different variables. If two features are highly correlated, they carry similar information, and including both may lead to redundancy and inefficiency in the model. By analyzing correlation, data scientists can streamline the feature set by removing redundant variables, leading to simpler, more interpretable, and often more effective ML models. Additionally, understanding the correlation between features and the

target variable assists in selecting the most relevant features, thus improving the model's performance.

For a visual representation of the interdependence of variables, in Python, it is suitable to show them in the form of a correlation matrix or a pair plot in the form of a scatter plot. These plots are easily implemented with the tools built into the **seaborn**, **matplotlib** and **pandas** libraries. The Pearson correlation matrix is calculated using the **pandas** library by calling the command **correlation_matrix = df.corr()**. The command is simply transformed into a Spearman or Kendall correlation by passing the required option as an argument to the previous command. It is common for the correlation level to be represented by different colour intensities, most often in blue-red heatmap variants. If the **seaborn** library is imported as **sns**, such a representation is achieved by calling the command **sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")** which, in addition to colour intensity, will display the correlation values as numerical values with two decimal places. The scatter plot joins the points in the coordinate plane for a pair of continuous variables, which gives insight into the type of interdependence (linear, non-linear, no relationship) as well as the direction of interdependence (positive, negative). The scatter plot is obtained by selecting the variables that you want to display on the graph and calling the following set of commands.

```
col = df[['var1', 'var2']]
sns.pairplot(col)
plt.show()
```

Additionally, the scatter plot can also be used to detect outliers, and unexpected observations, but also to assess the quality of regression models fitted to empirical data.

7. Introduction to machine learning algorithms

ML is centered around the concept of training algorithms on data sets, allowing them to make predictions or decisions without being explicitly programmed for specific tasks. The process relies on the training and testing sets, where the training set represents a large collection of data used to teach the model about the underlying patterns and relationships. This set represents a substantial portion of the total data, usually in the range of 70–80%. The testing set, consisting of the remaining 20–30% of the data, evaluates the model's performance on unseen data, ensuring that it generalizes well beyond the examples it was trained on. The ratio between these sets is important: an imbalance might lead to overfitting, where the model performs well on its training data but poorly on new, unseen data, or underfitting, where the model fails to learn adequately from the training data.

In Python, all ML algorithms must define a dependent variable (**y**) based on the input column **col** and a set of independent variables (**X**). In addition, it is necessary to divide the inputted empirical database into training and test data sets using some ratio **r**. If the dataset is imported in **df**, the previous steps are achieved with the following commands.

```
from sklearn.model_selection import train_test_split
X = df.drop('col', axis=1)
y = df['col']
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=r)
```

The next step is the model initialization with a set of defined hyperparameters (**param**), followed by the training phase, in which the model is applied to a defined set of variables (**X_train**, **y_train**). Please note that the hyperparameters set (**param**) may vary depending on the algorithm used. The procedure ends with a testing and prediction phase in which, based on the input set of independent variables (**X_test**), the output values of the algorithm are predicted, and they are placed in the variable **y_pred**, which is further used to calculate the quality of the formed algorithm, by comparing the values of **y_pred** with the values **y_test**.

7.1. Decision trees

DT is an intuitive and effective ML technique which segments the dataset into various branches based on input feature values (Breiman et al., 1984). By breaking down a dataset into smaller subsets while simultaneously developing an associated decision tree, the model is able to make predictions. The structure of a DT consists of nodes, branches, and leaves, where each node represents a feature (or attribute), each branch represents a decision (or rule), and each leaf represents an outcome.

The procedure for loading **DecisionTreeRegressor**, its initialization, training, and calculation of prediction values is given by the following set of Python commands.

```
from sklearn.tree import DecisionTreeRegressor
tree_regressor = DecisionTreeRegressor(param)
tree_regressor.fit(X_train, y_train)
y_pred = tree_regressor.predict(X_test)
```

7.2. Random forest

RF is an ensemble method that operates by constructing a multitude of DT at training time to output the mean prediction of the individual trees (Breiman, 2001a). Characterized by its randomness, each tree in a RF works on a random subset of data and considers a random set of features for splitting at each node, enhancing the model's ability to generalize and reduce overfitting.

To implement RF in Python, it is necessary to load the **RandomForestRegressor** function from the **sklearn.ensemble** library, followed by training and testing of the algorithm.

```
rf_regressor = RandomForestRegressor(param)
rf_regressor.fit(X_train, y_train)
y_pred = rf_regressor.predict(X_test)
```

7.3. Gradient boosting and stochastic gradient boosting

Boosting is an ensemble technique designed to enhance the accuracy of predictive models by learning from errors. It sequentially trains multiple weak models, typically decision trees, where each subsequent model aims to correct the mistakes made by its predecessors. The process involves assigning greater weight to the instances that were misclassified by earlier models, encouraging the new models to focus on difficult cases. This iterative correction helps in assembling a strong final model from several weaker ones.

GB is a specific implementation of the boosting technique that minimizes errors using a gradient descent algorithm (Friedman, 2000).

It builds trees one at a time, where each new tree helps to correct errors left by the previous ones. The method involves calculating the negative gradient of the loss function, which is akin to the error, and then constructing a new tree to predict these residuals. SGB, an extension of GB, introduces randomness into the process by training each new tree on a random subset of the data rather than the entire data set, which helps in reducing overfitting and improving model robustness (Friedman, 2002).

The primary distinction between GB and SGB resides in the incorporation of randomness in SGB. While GB systematically trains on all available data points, SGB adds an element of randomness by randomly sampling the training data for building each tree. This sampling without replacement resembles the process used in bagging methods but is applied within a boosting framework. This randomness not only makes SGB more robust against overfitting compared to GB but also contributes to a variance reduction in the final model, making SGB potentially more effective in handling diverse datasets.

Simple Python implementation of GB is given in the next program code.

```
from sklearn.ensemble import GradientBoostingRegressor
gb_regressor = GradientBoostingRegressor(param)
gb_regressor.fit(X_train, y_train)
y_pred = gb_regressor.predict(X_test)
```

The only difference between calling GB and SGB is that SGB has an additional parameter **subsample** to which it is necessary to assign a value that is less than 1.0, so the command for its initialization is **GradientBoostingRegressor(param, subsample)**. The rest of the code would be the same as for the GB algorithm.

7.4. Extreme Gradient Boosting

XGBoost represents an efficient and scalable implementation of the gradient boosting framework (Wade and Glynn, 2020). This approach facilitates parallel tree boosting and is capable of handling extensive datasets and complex feature dimensions. It brings several advantages, including the capability to process sparse data, providing a variety of tunable parameters that can be optimized for better performance, and offering an efficient implementation of the gradient boosting algorithm. The algorithm's ability to perform both sequential and parallel processing significantly accelerates the training process and enhances the accuracy of the resulting model.

As in the case of previously defined ML algorithms, for XGBoost it is necessary to load the library that contains the necessary functionalities for the implementation of the Extreme Gradient Boosting algorithm, after which the standard steps of training and testing follow, as shown in the following program code.

```
import xgboost as xgb
xgb_regressor = xgb.XGBRegressor(param)
xgb_regressor.fit(X_train, y_train)
y_pred = xgb_regressor.predict(X_test)
```

8. Evaluating model performance

In the evaluation of predictive models within statistical analysis and ML, various performance metrics are used to assess the accuracy, reliability, and applicability of developed models. These metrics serve as the foundation for understanding the differences between the predicted and actual outcomes, thus guiding researchers in refining and optimizing their predictive algorithms. Each metric offers a unique perspective on how well a model is performing, focusing on different aspects of the errors made by the predictions compared to actual values. Understanding these metrics is crucial for interpreting the results of a model, optimizing its performance, and ensuring it meets the specific

needs of a given application or research question (James et al., 2013). Mean Absolute Error (MAE) provides an intuitive measure by calculating the average magnitude of errors between predicted values and actual outcomes, without considering their direction. It represents the average error made by the model across all instances, offering a clear and direct interpretation of the model's prediction accuracy. MAE quantifies the average distance from the actual values, making it a robust indicator of model performance.

Mean Squared Error (MSE) extends upon MAE by squaring the individual errors before averaging them, amplifying the impact of larger errors. This characteristic makes MSE particularly useful in scenarios where large errors are especially undesirable or costly. By squaring the differences, MSE detects larger differences more severely than smaller ones, providing a more sensitive measure of model performance accuracy.

Mean Absolute Percentage Error (MAPE) shifts the focus from absolute error sizes to the relative errors made by the model, expressed as percentages. This metric is important to understand the error in proportion to the actual values. MAPE demonstrates the model's performance in terms of its average percentage deviation from the actual values, offering a scale-independent metric that is particularly useful when dealing with varying magnitudes of data points.

The coefficient of determination, denoted as R^2 , diverges from the aforementioned error metrics by providing a measure of the proportion of variance in the dependent variable that is predictable from the independent variables. R^2 offers a scale from 0 to 100%, where higher percentage values indicate a better fit between the model and the actual data. This metric is essential for evaluating the explanatory power of the model, as it quantifies how well the independent variables explain the variation in the dependent variable.

For these metrics, it is necessary to load the `numpy` library and import the functions for the required metrics from the `sklearn.metrics` library, i.e., `import mean_absolute_error, mean_squared_error, and r2_score`. All metrics imply the existence of real, empirical values of the dependent variable (`y_test`) and output values predicted/calculated with the applied algorithm (`y_pred`). After that, the required metrics are calculated by simply calling the commands.

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
mape = np.mean(np.abs((np.array(y_test) - np.array(y_pred)) /
    ↪ np.array(y_test))) * 100
r2 = r2_score(y_test, y_pred)
```

Finally, it is necessary to print the obtained values using the `print` command.

To display a plot for SHAP summary values, it is necessary to import the `shap` library into the Python environment. Then the variable `model` is assigned the value of one of the defined models from the set `dt_regressor, rf_regressor, gb_regressor, or xgb_regressor` for which the plot is to be displayed, and finally, the following series of commands is called.

```
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)
shap.summary_plot(shap_values, X_test, show=False)
plt.show()
```

If you want to display the SHAP plot for other models, repeat all these lines of code for the models you want to visualize.

9. Practical application: a case study

In psychology, it is interesting to identify factors influencing an individual's IS. Internal shame is a degrading emotional experience that is associated with avoidance tendencies, negative self-esteem, and feelings

of self-control (such as self-loathing) and it can affect a person's cognition and feelings in self-perception and individual functions. This study aims to investigate the correlation between IS and a set of independent variables: emotional competency (EC), distress tolerance (DIST), cognitive flexibility (CFL), alexithymia (ALEX), and childhood trauma (TRAU). The central research questions guiding this inquiry are: *Is there a mutual relationship between IS and other variables? How do TRAU, ALEX, CFL, DIST, and EC predict the numerical continuous value of an individual's IS? Which ML algorithm is the most convenient to predict IS?*

To address these research questions, the following steps were taken.

- 1. **Data Preprocessing:** The dataset was first cleaned and prepared. This involved handling missing values using the `pandas` library's functions, scaling the data using `MinMaxScaler` from `sklearn`, and identifying and removing outliers based on the IQR method.
- 2. **Feature Selection:** Correlations between variables were visualized using a Pearson correlation matrix to understand the relationships between the dependent variable (IS) and independent variables (EC, DIST, CFL, ALEX, TRAU). The pairwise relationships were examined using a `Pairplot` visualization.
- 3. **Model Training:** The data was split into training and test sets using an 80-20 ratio. ML regression models `DT, RF, GB, SGB, and XGBoost` were trained on the training set.
- 4. **Model Evaluation:** Each model's performance was evaluated using `MSE, MAE, MAPE, and  $R^2$` . The `SHapley Additive exPlanations (SHAP)` values were calculated to interpret the contribution of each feature.

To normalize the dataset, the `MinMax` scaler was employed, ensuring all features contributed equally by scaling them to a range between 0 and 1. The set of parameters and their values that were necessary to run the ML algorithms (`DT, RF, GB, SGB, and XGBoost`) are given in [Table 1](#). The test size parameter, set to 0.2, denotes that 20% of the dataset is allocated for testing, with the remaining 80% used for training. The random state parameter is set to 1, which controls the shuffling of the data before the split. This setting ensures that the split is reproducible, meaning the exact same training and testing datasets can be recreated across different runs, allowing for consistent comparison of model performance. Minimum samples split refers to the minimum number of samples required to split an internal node in a tree (set to 8), while minimum samples leaf is the minimum number of samples required to be at a leaf node (set to 4). Maximum depth limits the maximum depth or levels of the tree to prevent overfitting. The number of estimators is relevant for ensemble methods: `RF, GB, SGB, and XGBoost`, indicating the number of trees to be used. The learning rate controls how much each additional tree influences the overall prediction in boosting, reducing the risk of overfitting by making the training process more gradual. A lower rate requires more trees but can lead to better models. Subsample ratio affects the fraction of samples used for fitting each tree, where 1.0 means using all samples (no subsampling), while 0.9 means using 90% of samples, introducing more randomness and potentially reducing overfitting. Column ratio in `XGBoost` determines the fraction of features (columns) to be randomly sampled for

Table 1
Set of hyperparameters for ML algorithms.

Hyperparameter	DT	RF	GB	SGB	XGBoost
Test size	0.2	0.2	0.2	0.2	0.2
Random state	1	1	1	1	1
Minimum samples split	8	*	*	*	*
Minimum samples leaf	4	4	*	*	*
Maximum depth	5	5	4	4	5
Number of estimators	*	100	100	100	500
Learning rate	*	*	0.05	0.05	0.015
Subsample ratio	*	*	1.0	0.9	0.8
Column ratio	*	*	*	*	0.8
α	*	*	*	*	0.1
λ	*	*	*	*	1.5

```

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
↳ GradientBoostingRegressor
import xgboost as xgb
from sklearn.metrics import mean_squared_error,
↳ mean_absolute_error, mean_absolute_percentage_error,
↳ r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import shap

# Load dataset
df = pd.read_csv('data.csv')

# Check for NaN values
No_NaN = df.isna().any().any()
print(f"DataFrame contains NaN values: {No_NaN}")

# Data scaling
scaler = MinMaxScaler()
df = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

# Show pairplot
sns.pairplot(df, plot_kws={'color': 'green'},
↳ diag_kws={'color': 'green'})

# Pearson correlation matrix
corr_matrix = df.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='RdYlGn', fmt=".2f")
plt.show()

# Remove outliers

```

```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
outliers_mask = ((df < (Q1 - 1.5 * IQR)) | (df > (Q3 + 1.5 *
    ↳ IQR))).any(axis=1)
df = df[~outliers_mask]

X = df.drop('IS', axis=1) # Independent variable set
y = df['IS'] # Dependent variable

# Splitting the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
    ↳ test_size=0.2, random_state=1)

# Training
dt_regressor = DecisionTreeRegressor(min_samples_split=8,
    ↳ min_samples_leaf=4, max_depth=5)
dt_regressor.fit(X_train, y_train)
rf_regressor = RandomForestRegressor(n_estimators=100,
    ↳ max_depth=5, min_samples_leaf=4)
rf_regressor.fit(X_train, y_train)
gbm_regressor = GradientBoostingRegressor(n_estimators=100,
    ↳ learning_rate=0.05, subsample=1.0, max_depth=4)
gbm_regressor.fit(X_train, y_train)
sgbm_regressor = GradientBoostingRegressor(n_estimators=100,
    ↳ learning_rate=0.05, subsample=0.9, max_depth=4)
sgbm_regressor.fit(X_train, y_train)
xgb_regressor = xgb.XGBRegressor(n_estimators=500,
    ↳ learning_rate=0.015, subsample=0.8, colsample_bytree=0.8,
    ↳ max_depth=5, reg_alpha=0.1, reg_lambda=1.5)
xgb_regressor.fit(X_train, y_train)

# Predictions
y_pred_dt = dt_regressor.predict(X_test)
y_pred_rf = rf_regressor.predict(X_test)
y_pred_gbm = gbm_regressor.predict(X_test)
y_pred_sgbm = sgbm_regressor.predict(X_test)
y_pred_xgb = xgb_regressor.predict(X_test)
```

```

# Evaluation Metrics
mse_dt = mean_squared_error(y_test, y_pred_dt)
mae_dt = mean_absolute_error(y_test, y_pred_dt)
mape_dt = mean_absolute_percentage_error(y_test, y_pred_dt)
r2_dt = r2_score(y_test, y_pred_dt)
mse_rf = mean_squared_error(y_test, y_pred_rf)
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mape_rf = mean_absolute_percentage_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)
mse_gbm = mean_squared_error(y_test, y_pred_gbm)
mae_gbm = mean_absolute_error(y_test, y_pred_gbm)
mape_gbm = mean_absolute_percentage_error(y_test, y_pred_gbm)
r2_gbm = r2_score(y_test, y_pred_gbm)
mse_sgbm = mean_squared_error(y_test, y_pred_sgbm)
mae_sgbm = mean_absolute_error(y_test, y_pred_sgbm)
mape_sgbm = mean_absolute_percentage_error(y_test, y_pred_sgbm)
r2_sgbm = r2_score(y_test, y_pred_sgbm)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mape_xgb = mean_absolute_percentage_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

print('DT: ', mse_dt, mae_dt, mape_dt, r2_dt)
print('RF: ', mse_rf, mae_rf, mape_rf, r2_rf)
print('GBM: ', mse_gbm, mae_gbm, mape_gbm, r2_gbm)
print('SGBM: ', mse_sgbm, mae_sgbm, mape_sgbm, r2_sgbm)
print('XGB: ', mse_xgb, mae_xgb, mape_xgb, r2_xgb)

model = xgb_regressor
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)
shap.summary_plot(shap_values, X_test, show=False)
plt.show()

```

each tree, enhancing diversity among the trees. Parameters α and λ are regularization parameters in XGBoost. Parameter α is the L1 regularization term (equivalent to Lasso regression) and helps with feature selection. Parameter λ is the L2 regularization term (equivalent to Ridge regression) which reduces overfitting by making the model smoother.

The final Python code that covers all the above mentioned algorithms and the necessary steps to answer the research question is given below.

The Pearson Correlation Matrix illustrating the linear relationships between IS and the independent variables is displayed in Fig. 1. Each cell in the matrix represents the correlation coefficient between the pair of

variables, with values ranging from -1 (perfect negative correlation) to 1 (perfect positive correlation). This visual helps identify which variables are most strongly related to IS, guiding further feature selection and analysis.

The Pairplot providing a grid of scatter plots showing the pairwise relationships between IS and EC, DIST, CFL, ALEX, and TRAU is displayed in Fig. 2. The diagonal elements display the distribution of each variable, while the off-diagonal elements show scatter plots for each pair of variables, offering insights into their potential linear or non-linear relationships. This plot is important for detecting patterns, trends, and possible outliers that may influence the model's predictions.

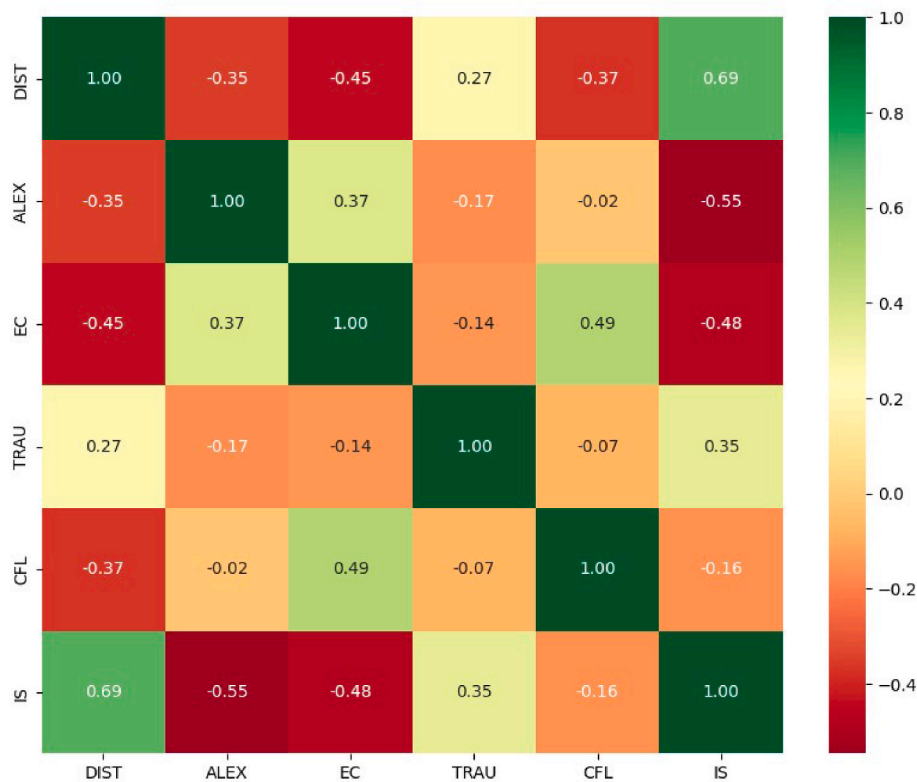


Fig. 1. Pearson correlation.

SHAP values which indicate how much each feature value contributes, either positively or negatively, to the output variable (IS) in comparison to the average prediction of the regression model are shown in Fig. 3. Each dot represents a SHAP value for a feature and an instance, with colors indicating the feature value (green for high and red for low). The x-axis shows the SHAP value magnitude, reflecting the impact on the prediction. This plot helps in understanding the importance and effect of each feature, providing a transparent interpretation of the model’s decisions. The SHAP values are shown as an example only for XGBoost in the program code and in Fig. 3, while for the other models it can be generated analogously. In the provided Python code, for each of the algorithms, it is possible to further display additional visualizations, such as the Learning Curve, Prediction Error, Residuals Plot, QQ plot, and Feature importance.

ML offers significant advantages in the field of psychology by providing a pathway towards becoming a more predictive science, complementing traditional explanatory approaches. The emphasis on explaining causal mechanisms of behavior in psychology has often resulted in theories that have limited predictive power regarding future behaviors. ML, with its focus on predicting unobserved outcomes based on patterns in large datasets, proposes a shift towards prioritizing prediction over explanation. This shift enables greater understanding and accuracy in forecasting behaviors, thereby addressing a critical gap in psychological research.

10. Conclusions

ML offers transformative potential in psychology by enabling analysis and understanding of complex human behaviors and mental processes. Through its capacity to process and analyze extensive amounts of data, ML provides psychologists with insights and precise information into data patterns and trends. This can lead to the development of more effective, personalized therapeutic techniques and interventions.

The application of ML techniques and principles can significantly enhance the predictive accuracy of psychological models. By leveraging these techniques, psychology can benefit from the ability to generalize findings more effectively to new, unobserved instances. This is particularly relevant in light of the replication crisis, where many psychological findings have failed to be reproduced, often due to overfitting and reliance on small sample sizes that do not adequately represent the broader population.

ML encourages the use of predictive accuracy as a common metric to evaluate and compare models. This focus on tangible outcomes helps in bridging theoretical insights with practical applications, fostering developments that are both theoretically innovative and empirically robust. Thus, integrating ML into psychological research promises to enhance the field’s methodological rigor, improve the reliability of its findings, and ultimately, contribute to a deeper understanding of human behavior.

This study offers several key strengths. It provides a comprehensive, step-by-step tutorial designed for psychologists, bridging the gap

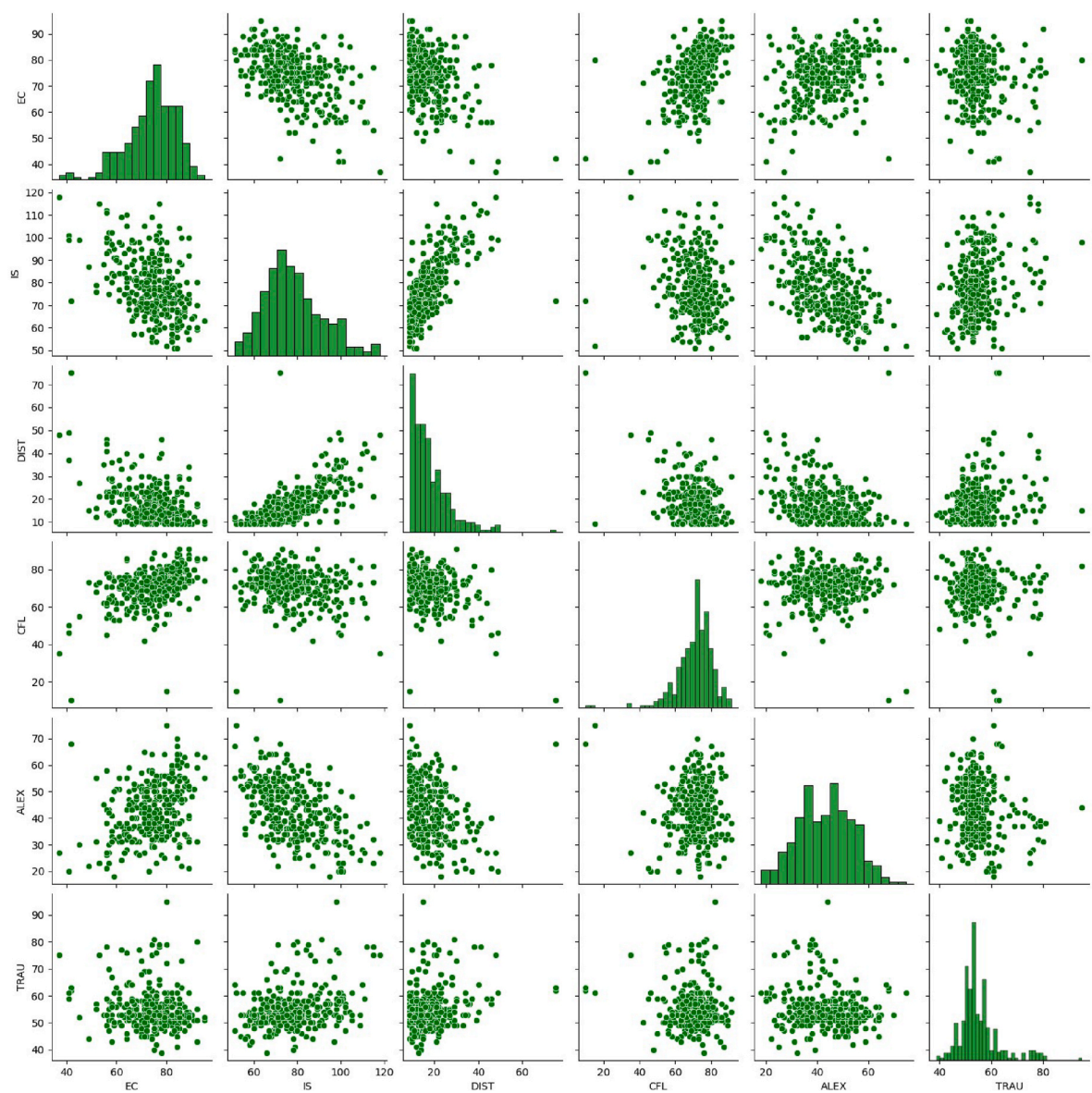


Fig. 2. Pairplot.

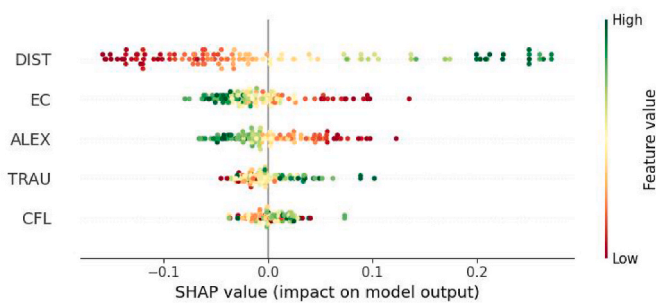


Fig. 3. Shap values.

between theoretical knowledge and practical application of ML techniques. The tutorial includes a detailed case study focusing on internal shame, which serves as a practical example of how various ML regression methods can be applied to real-world psychological data. The use of Python ensures that the tools and techniques presented are both powerful and user-friendly, enabling psychologists from diverse backgrounds to adopt and implement ML methodologies effectively. By emphasizing data preprocessing, feature selection, and model evaluation, the tutorial equips researchers with the skills needed to conduct robust and reproducible analyses, thereby enhancing the reliability and impact of psychological research.

Despite its strengths, this study has certain limitations. The tutorial is designed primarily for psychologists with limited programming experience, and may not be fully suitable for advanced users looking for more in-depth technical details or novel ML methodologies. The case study focuses on a specific psychological construct (internal shame), which, while illustrative, may limit the generalizability of the techniques to other psychological phenomena without additional adaptation. The reliance on Python and its libraries, though beneficial for accessibility, may pose challenges for those accustomed to other programming languages or statistical software. This study does not extensively cover the ethical implications of ML in psychological research, an area that is increasingly important as data-driven methods become more prevalent. Future work could address these limitations by incorporating more advanced ML techniques, expanding the range of psychological constructs analyzed, and providing a broader discussion on the ethical use of ML in psychology.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRedit authorship contribution statement

Nataša Kovač: Writing – review & editing, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Kruna Ratković:** Writing – review & editing, Writing – original draft, Validation, Formal analysis, Conceptualization. **Hojjatollah Farahani:** Writing – original draft, Validation, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Peter Watson:** Writing – review & editing, Validation, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial

interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Agresti, A., 2012. *Categorical Data Analysis*, ume 792. John Wiley & Sons.
- Aguinis, H., Gottfredson, R.K., Joo, H., 2013. Best-practice recommendations for defining, identifying, and handling outliers. *Organ. Res. Methods* 16 (2), 270–301.
- Bennett, M., Hayes, K., Kleczyk, E.J., Mehta, R., 2022. Similarities and Differences between Machine Learning and Traditional Advanced Statistical Modeling in Healthcare Analytics arXiv preprint arXiv:2201.02469.
- Breiman, L., 2001a. Random forests. *Mach. Learn.* 45, 5–32.
- Breiman, L., 2001b. Statistical modeling: the two cultures (with comments and a rejoinder by the author). *Stat. Sci.* 16 (3), 199–231.
- Breiman, L., Friedman, J., Stone, C.J., A, O.R., 1984. *Classification and Regression Trees*. Chapman and Hall/CRC.
- Bzdok, D., Altman, N., Krzywinski, M., 2018. Statistics versus machine learning. *Nat. Methods* 15, 233–234.
- d'Aspremont, A., Ghaoui, L., Jordan, M., Lanckriet, G., 2004. A direct formulation for sparse PCA using semidefinite programming. *Adv. Neural Inf. Process. Syst.* 17.
- Famili, A., Shen, W.-M., Weber, R., Simoudis, E., 1997. Data preprocessing and intelligent data analysis. *Intell. Data Anal.* 1 (1), 3–23.
- Friedman, J.H., 2000. Greedy function approximation: a gradient boosting machine. *Ann. Stat.* 29, 1189–1232.
- Friedman, J.H., 2002. Stochastic gradient boosting. *Comput. Stat. Data Anal.* 38 (4), 367–378.
- Ghosh, D., Vogt, A., 2012. Outliers: an evaluation of methodologies. In: *Joint statistical meetings*, 12, pp. 3455–3460.
- Ioannidis, J.P., Tarone, R., McLaughlin, J.K., 2011. The false-positive to false-negative ratio in epidemiologic studies. *Epidemiology* 22 (4), 450–456.
- Jacobucci, R., Grimm, K.J., 2020. Machine learning and psychological research: the unexplored effect of measurement. *Perspect. Psychol. Sci.* 15 (3), 809–816.
- Jadhav, A., Dhaulakhadi, D., Shandilya, S.K., Malviya, L., Mewada, A., 2023. Data transformation: a preprocessing stage in machine learning regression problems. In: *Artificial Intelligence Techniques in Power Systems Operations and Analysis*. Auerbach Publications, pp. 183–194.
- James, G., Witten, D., Hastie, T., Tibshirani, R., 2013. *An introduction to statistical learning*, Vol. 112. Springer, New York.
- Kotsiantis, S.B., Kanellopoulos, D., Pintelas, P.E., 2006. Data preprocessing for supervised learning. *Int. J. Comput. Sci.* 1 (2), 111–117.
- Koul, A., Becchio, C., Cavallo, A., 2018. Predpsych: a toolbox for predictive machine learning-based approach in experimental psychology research. *Behav. Res. Methods* 50, 1657–1672.
- Lutz, M., 2001. *Programming Python*. O'Reilly Media, Inc.
- Martelli, A., Ravenscroft, A.M., Holden, S., McGuire, P., 2023. *Python in a Nutshell*. O'Reilly Media, Inc.
- Orrù, G., Monaro, M., Conversano, C., Gemignani, A., Sartori, G., 2020. Machine learning in psychometrics and psychological research. *Front. Psychol.* 10, 492685.
- Peeters, M., Zondervan-Zwijenburg, M., Vink, G., Van de Schoot, R., 2015. How to handle missing data: a comparison of different approaches. *Eur. J. Dev. Psychol.* 12 (4), 377–394.
- Rolon-Mérette, D., Ross, M., Rolon-Mérette, T., Church, K., 2016. Introduction to Anaconda and Python: installation and setup. *Quant. Methods Psychol* 16 (5), S3–S11.
- Rosenbusch, H., Soldner, F., Evans, A.M., Zeelenberg, M., 2021. Supervised machine learning methods in psychology: a practical introduction with annotated R code. *Social and Personality Psychology Compass* 15 (2), e12579.
- Shen, H., Huang, J.Z., 2008. Sparse principal component analysis via regularized low rank matrix approximation. *J. Multivariate Anal.* 99 (6), 1015–1034.
- Toomey, D., 2016. *Learning Jupyter*. Packt Publishing Ltd.
- Vidaurre, D., Bielza, C., Larranaga, P., 2013. A survey of L1 regression. *Int. Stat. Rev.* 81 (3), 361–387.
- Wade, C., Glynn, K., 2020. *Hands-On Gradient Boosting with XGBoost and Scikit-Learn : Perform Accessible Machine Learning and Extreme Gradient Boosting with Python*. Packt Publishing.
- Yarkoni, T., Westfall, J., 2017. Choosing prediction over explanation in psychology: lessons from machine learning. *Perspect. Psychol. Sci.* 12 (6), 1100–1122.
- Zou, H., Hastie, T., Tibshirani, R., 2006. Sparse principal component analysis. *J. Comput. Graph Stat.* 15 (2), 265–286.